



CareQueue

Project Documentation and Interview Preparation Guide

For Frontend Developer Fresher HR and Technical Rounds

One-line project summary

CareQueue is a React-based healthcare appointment booking frontend where users can sign up, log in, search diagnostic services or doctor consultations, book appointments, and manage their bookings.

Item	Details
Project type	Frontend single page application
Domain	Healthcare appointment and diagnostic booking
Main users	Patients who want to book tests, scans, X-rays, ECG, echocardiogram, and consultations
Deployment	Vercel, connected to GitHub
Important note	This is a frontend prototype. Authentication and booking persistence are simulated using browser localStorage.

1. Project Introduction

CareQueue is a healthcare booking web application designed to reduce waiting time and simplify the process of finding diagnostic centres and hospital consultations. The user can create an account, log in, browse services, view matching centres or doctors, book an appointment, and later view or cancel the appointment.

The project is built as a frontend-first prototype. Instead of using a backend server, it uses mock data files and browser localStorage to simulate users, login state, and saved bookings. This makes the project easy to demonstrate in an interview while still showing core frontend skills such as routing, reusable components, state management, forms, conditional rendering, and deployment.

Problem Statement

Patients often need to compare centres, check available services, and book diagnostic tests without calling multiple places. CareQueue solves this by providing a simple interface where users can select a healthcare service and quickly find relevant centres or doctors.

Objectives

- Build a clean and responsive healthcare booking interface.
- Allow users to sign up, log in, and maintain a session in the browser.
- Show diagnostic centres based on the selected service.
- Support doctor consultations using structured hospital and doctor data.
- Collect appointment details through controlled React forms.
- Store booking data locally and show it on the My Appointments page.
- Deploy the application publicly using GitHub and Vercel.

2. Technology Stack

Technology	Use in CareQueue
React	Used to build the user interface with components such as Navbar, AuthPage, SearchResults, BookingForm, and MyAppointments.
Vite	Used as the frontend build tool and development server. It provides fast startup, fast refresh, and production bundling.
React Router DOM	Used for client-side navigation between pages like /, /search, /book-appointments, /booking-form, /appointments, and /about.
Tailwind CSS	Used for utility-first styling, responsive layouts, spacing, colors, grids, and form design.
JavaScript ES Modules	Used to organize components and data files with import/export.
localStorage	Used to simulate users, current login session, and appointment bookings in the browser.
ESLint	Used to catch unused variables and React code quality issues before deployment.
Vercel	Used to deploy the Vite build output from GitHub.
GitHub	Used as the version control and source repository connected to Vercel.

Interview wording

I used React and Vite for the frontend, React Router for page navigation, Tailwind CSS for styling, and localStorage to simulate persistence for this frontend prototype. The app is deployed on Vercel through GitHub.

3. Main Features

Feature	Description
Login and signup	Users can register with email and password. Details are stored in the browser under the users key.
Protected routes	If the user is not logged in, routes show the AuthPage instead of the actual page.
Landing page	Displays a hero section, healthcare service highlights, quick stats, and call-to-action buttons.
Service browsing	Users can browse blood tests, scans, ECG, echocardiogram, X-rays, and doctor consultations.
Search results	The app filters diagnostic centres or doctor consultation data based on the selected service.
Booking form	Users enter name, age, mobile number, appointment date, timing, and patient information.
Booking history	Confirmed appointments are stored and displayed for the currently logged-in user.
Cancel appointment	The appointment status can be changed from Confirmed to Cancelled.
Deployment-ready routing	A Vercel rewrite config sends all routes to index.html so React Router routes work after refresh.

4. User Flow

The main flow of the application is simple and can be explained clearly in interviews:

- 1 The user opens the CareQueue website.
- 2 If not logged in, the AuthPage is shown with Login and Signup tabs.
- 3 After signup or login, the user is taken to the home page.
- 4 The user selects a service such as blood tests, scans, ECG, X-ray, or a consultation.
- 5 The app navigates to /search with the selected service in the URL query string.
- 6 SearchResults reads the service from the URL and filters matching centres or doctors.
- 7 The user selects a centre or doctor and moves to the BookingForm page.
- 8 BookingForm displays service details, price, reviews, doctor details if applicable, and the appointment form.
- 9 On submit, a booking object is stored in localStorage under carequeue_bookings.
- 10 The user is redirected to My Appointments, where their bookings are filtered by the logged-in email.

Flow Diagram

Open App -> Signup/Login -> Landing Page -> Select Service -> Search Results -> Booking Form -> Save Booking -> My Appointments -> Cancel if needed

5. Application Architecture

CareQueue follows a component-based frontend architecture. Each major screen is a React component, and shared layout pieces such as Navbar and Footer are reused across pages. Data is separated into data files for cleaner code organization.

Area	Files / Responsibility
Entry point	src/main.jsx creates the React root and renders App inside StrictMode.
Routing	src/App.jsx defines routes and checks login status before showing protected pages.
Authentication state	src/components/AuthContext.jsx stores isLoggedIn state and exposes login/logout functions.
Auth UI	src/components/AuthPage.jsx handles Login and Signup tabs.
Home page	src/components/LandingPage.jsx shows the main dashboard-style landing page.
Service selection	src/components/BookAppointments.jsx lists available diagnostic and consultation services.
Search results	src/components/SearchResults.jsx filters centres or doctors based on URL query params.
Booking	src/components/BookingForm.jsx collects appointment details and writes bookings to localStorage.
Appointments	src/components/MyAppointments.jsx reads and filters bookings for the current user.
Mock pricing data	src/data/pricing.js maps centre/service combinations to prices.
Mock consultation data	src/data/consultations.js stores consultation types, hospitals, doctors, and reviews.

6. Routing Explanation

The app uses BrowserRouter from React Router DOM. The AppContent component reads isLoggedIn from the auth context and conditionally renders either the requested page or AuthPage. This is a simple protected-route pattern suitable for a frontend prototype.

Route	Purpose
/	Shows LandingPage after login, otherwise AuthPage.
/search	Shows SearchResults for the selected service query.
/book-appointments	Shows the service catalogue page.
/booking-form	Shows the detailed booking form for a chosen centre/doctor.
/appointments	Shows the logged-in user's bookings.
/about	Shows the about/information page.

Why vercel.json is needed

Because BrowserRouter handles routes in the browser, refreshing /appointments directly can fail on static hosts unless the host sends every route back to index.html. The vercel.json rewrite fixes that for Vercel.

7. Authentication and State Management

CareQueue uses React state and Context API for login status. The AuthProvider stores isLoggedIn and exposes login and logout functions. When login succeeds, the current user is stored in localStorage using the currentUser key.

localStorage key	What it stores
users	Array of registered users from the signup form.
currentUser	The currently logged-in user's details.
carequeue_bookings	Array of appointment booking records.

Important technical explanation

- Signup reads existing users, checks duplicate email, then saves the new user.
- Login compares entered email/password with saved users.
- AuthProvider checks localStorage on page load to restore login state.
- The app uses conditional rendering to protect pages from unauthenticated users.
- This is not real secure authentication; it is a frontend simulation for demo purposes.

Interview honesty

Say this clearly: For this project I used localStorage to simulate authentication because my focus was frontend flow. In a production app I would use a backend with hashed passwords, JWT or sessions, and a real database.

8. Search and Booking Logic

SearchResults works in two modes: diagnostic services and doctor consultations.

Mode	How it works
Diagnostic services	The selected service is compared with each centre's services array. Matching centres are displayed with image, rating, location, phone, services, and price.
Doctor consultations	If the selected service matches CONSULTATION_TYPES, the app looks inside HOSPITAL_DOCTORS and displays hospitals with matching doctors.
Price display	getPriceDetails(centerName, serviceName) returns a centre-specific price or a default price.
Booking navigation	Clicking a result sends centre, service, and optionally doctor details through URL query params to BookingForm.

BookingForm responsibilities

- Reads centre, service, and doctor from URL search parameters.
- Displays centre/hospital information, rating, reviews, price, and report timing.
- Uses controlled form fields for appointment data.
- Supports booking for myself or someone else.
- Creates a booking object with id, centreName, price, userEmail, status, and bookedAt.
- Saves the booking to carequeue_bookings in localStorage.
- Shows a confirmation message and redirects to /appointments.

9. Appointment Management

The MyAppointments component displays only the bookings that belong to the logged-in user. It reads currentUser from localStorage, loads all bookings from carequeue_bookings, and filters them by userEmail.

Action	Implementation
View appointments	Filter all saved bookings using the current user's email.
Summary badges	Show total, confirmed, and cancelled appointment counts.
Cancel appointment	Update the matching booking status to Cancelled and save the updated array.
Empty state	If no bookings exist, show a message and link to book an appointment.

10. Deployment Explanation

The project is deployed on Vercel from a GitHub repository. Vercel installs dependencies using npm install, builds the project using npm run build, and serves the dist folder generated by Vite.

Vercel setting	Value
Root Directory	React/CareQueue_project/carequeue
Framework / Preset	Vite
Install Command	npm install
Build Command	npm run build
Output Directory	dist
Routing config	vercel.json rewrites all paths to /index.html

How updates work

When changes are pushed to GitHub, Vercel automatically creates a new deployment. Manual redeploy is only needed if a build fails, settings change, or the same commit needs to be rebuilt.

11. HR Round Explanation

30-second answer

CareQueue is a healthcare appointment booking web app that I built using React. The main goal is to help users book diagnostic tests, scans, X-rays, ECG, echocardiogram, and doctor consultations from one place. Users can sign up, log in, search services, book appointments, and view or cancel bookings. I deployed it on Vercel, so it is available as a live project.

1-minute answer

In CareQueue, I focused on building a practical frontend application instead of only a static website. I implemented routing with React Router, created reusable components, used forms for booking data, and stored users and bookings in localStorage to simulate backend persistence. The project helped me understand component structure, state management, conditional rendering, form handling, and deployment. As a next step, I would connect it to a backend and database for real authentication and real appointment management.

HR strengths to mention

- I chose a real-world healthcare problem instead of a simple clone.
- I completed the full user journey from login to booking history.
- I deployed the project and fixed deployment configuration issues.
- I understand the limitations and know how to improve it with backend integration.

12. Technical Round Explanation

How I built it

- 1 I started with a Vite React setup for a fast development environment.
- 2 I created separate components for authentication, landing page, service listing, search results, booking form, and appointments.
- 3 I used React Router to create multiple client-side pages.
- 4 I created an AuthContext to share login/logout state across the application.
- 5 I used localStorage to store registered users, current user, and bookings.
- 6 I created separate data files for pricing and consultation data so the UI code stays cleaner.
- 7 I used conditional rendering to display diagnostic centres or doctor consultation results depending on the selected service.
- 8 I deployed the production build to Vercel and configured SPA routing using vercel.json.

Technical keywords you should be ready to explain

Keyword	Simple explanation
SPA	A single page application loads one HTML page and changes views using JavaScript routing.
Component	A reusable UI block in React, such as Navbar or BookingForm.
Props	Data passed from one component to another.

State	Data that changes inside a component, such as form fields or selected tab.
Context API	A way to share data like login state without passing props manually through many components.
Controlled form	A form where input values are stored in React state and updated through onChange.
localStorage	Browser storage used to persist small data even after refresh.
Client-side routing	Navigation handled by React Router without requesting a new HTML page from the server.

13. Common Interview Questions

Question	Good answer
Why did you build CareQueue?	I wanted to build a real-world frontend app where users complete an actual workflow: login, search, booking, and appointment management.
Why React?	React helps break the UI into reusable components and manage dynamic data such as forms, login state, and filtered search results.
Why Vite?	Vite gives a faster development server and optimized production build for modern frontend projects.
How does login work?	Signup saves users in localStorage. Login checks the entered credentials against stored users and saves currentUser on success.
Is this secure authentication?	No. It is a frontend prototype. In production I would use backend authentication, hashed passwords, JWT/session handling, and HTTPS APIs.
How are appointments stored?	Appointments are stored in localStorage as an array under carequeue_bookings. Each booking includes userEmail, service, centre, date, timing, price, and status.
How does search work?	The selected service is passed in the URL query string. SearchResults reads it and filters mock centre data or hospital doctor data.
How did you deploy it?	I pushed the code to GitHub, imported it into Vercel, set the correct root directory, and used Vite build settings.
What challenge did you face?	During deployment, Vercel first failed because it looked for package.json in the wrong folder. I fixed it by setting the correct root directory.
What would you improve next?	I would add a backend, database, real authentication, stronger validation, admin dashboard, appointment availability, and API-based centre data.

14. Limitations and Future Scope

Current limitations

- Authentication is stored in localStorage, so it is not secure for production.
- Appointments are stored only in the browser, so they do not sync across devices.
- Diagnostic centres, hospitals, doctors, prices, and reviews are mock data.
- There is no admin panel to manage appointments or centres.
- There is no payment gateway or real-time slot availability.

- Medical data privacy and compliance are not implemented because this is a frontend prototype.

Future enhancements

- Add Node.js/Express or Firebase backend.
- Use MongoDB, PostgreSQL, or Firebase Firestore for persistent data.
- Implement secure authentication using hashed passwords and JWT/sessions.
- Create an admin dashboard for centres to manage appointments.
- Add real slot availability, rescheduling, email/SMS confirmation, and payment flow.
- Improve form validation with better error messages.
- Add automated tests for forms, routing, and booking logic.
- Add API integration for real hospitals, centres, and doctor listings.

15. Resume and Portfolio Points

Resume bullet examples

- Built CareQueue, a React and Vite healthcare booking SPA with authentication flow, service search, appointment booking, and booking management.
- Implemented client-side routing using React Router and protected route rendering based on authentication state.
- Designed reusable React components for landing page, service listing, search results, booking forms, and appointment cards.
- Used localStorage to simulate user sessions and appointment persistence in a frontend prototype.
- Deployed the app on Vercel with correct Vite build settings and SPA rewrite configuration.

Short portfolio description

CareQueue is a healthcare appointment booking frontend built with React, Vite, React Router, and Tailwind CSS. It allows users to sign up, log in, search diagnostic services and doctor consultations, book appointments, and manage bookings. The project is deployed on Vercel and demonstrates practical frontend skills through a complete user journey.

16. Final Study Checklist

- Practice the 30-second and 1-minute explanation aloud.
- Be ready to explain the difference between frontend prototype and production app.
- Know the localStorage keys: users, currentUser, and carequeue_bookings.
- Understand how React Router query params are used for service and booking details.
- Open the live Vercel link before interview and check the main flow.
- Keep GitHub and live demo links ready.
- Do not claim real backend, real database, or real secure authentication.
- Explain future improvements confidently.

Best closing line in interview

This project helped me understand how to convert a real-world idea into a working frontend application with routing, forms, state, conditional rendering, local persistence, and deployment.